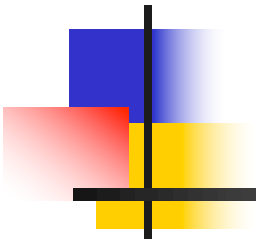
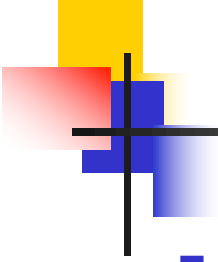


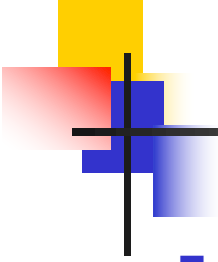
Tema 5: Desarrollo Basado en Componentes con React





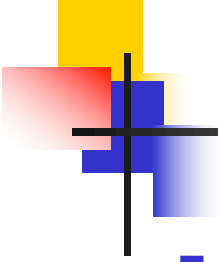
Índice

- Desarrollo basado en componentes
- Introducción a React
- Vite
- Desarrollo iterativo de un sencillo gestor de TODOs
- Virtual DOM
- Inmutabilidad



Desarrollo basado en componentes

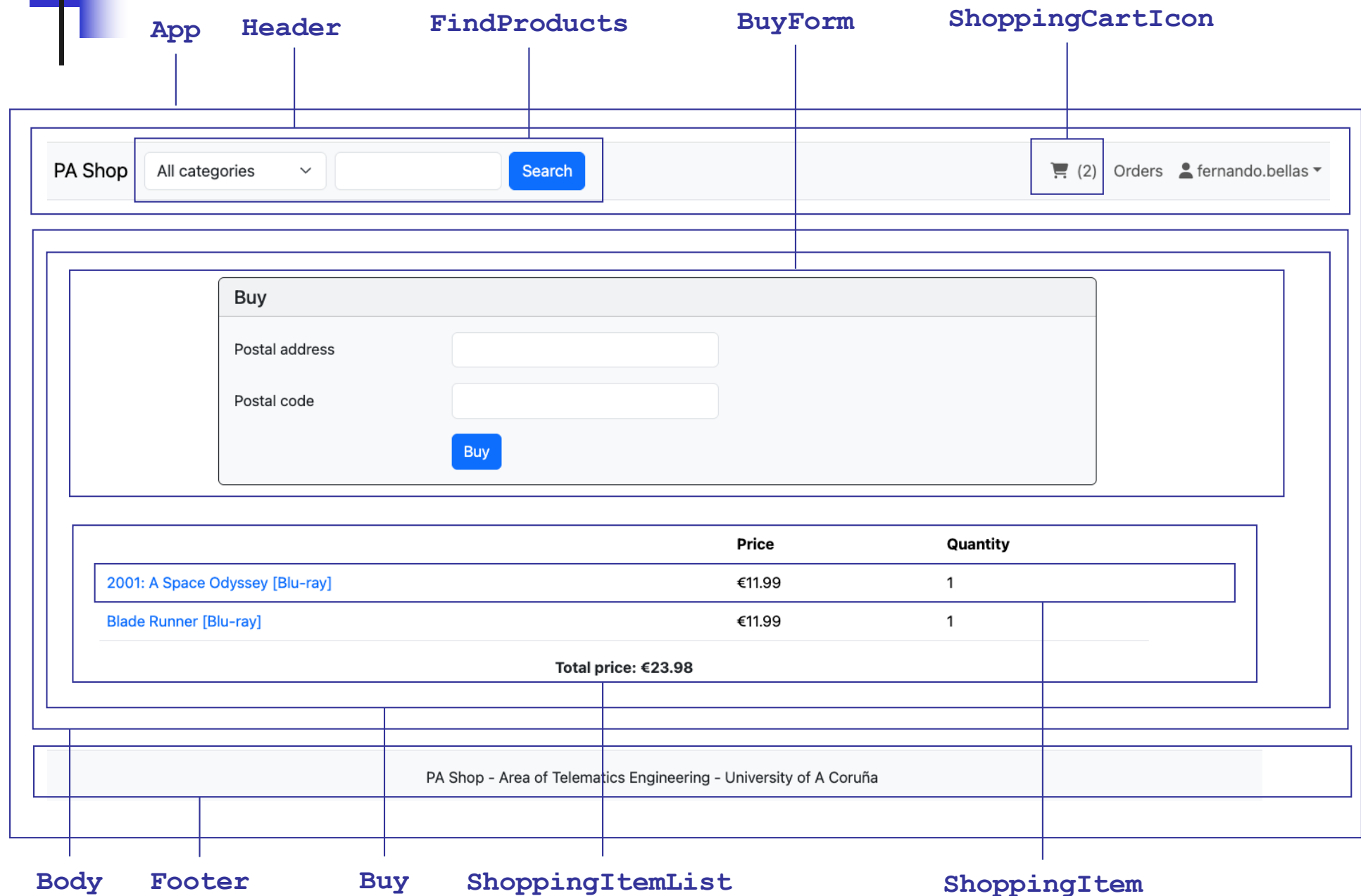
- El desarrollo basado en componentes es un paradigma bastante habitual en la implementación de la capa IU
- Bajo este paradigma, la capa IU se descompone en partes más pequeñas, llamadas **componentes**
- Cada componente genera una parte de la IU y responde a los eventos de usuario relativos a esa parte
- Ventajas
 - Divide y vencerás: se aborda el desarrollo de una IU compleja mediante la implementación de componentes pequeños
 - Reusabilidad: algunos componentes serán reusables, dentro de una aplicación, e incluso entre aplicaciones



Desarrollo basado en componentes

- Ejemplos de plataformas de desarrollo de IU que siguen el paradigma basado en componentes
 - Los SDKs de las plataformas de desarrollo de aplicaciones nativas, tanto para desktop como móvil
 - Frameworks de desarrollo de aplicaciones web SPA (Angular, React y Vue.js)
 - Algunos frameworks de desarrollo de aplicaciones web del lado servidor (e.g. Tapestry, Wicket, etc.)

Desarrollo basado en componentes – Divide y vencerás



Desarrollo basado en componentes – Reusabilidad

- `ShoppingItemList` se usa en la compra (diapositiva anterior), en la visualización de los ítems del carrito (esta diapositiva) ...

PA Shop All categories Search (2) Orders fernando.bellas

	Price	Quantity	
2001: A Space Odyssey [Blu-ray]	€11.99	1	Save
Blade Runner [Blu-ray]	€11.99	1	Save
Total price: €23.98			
Buy			

PA Shop - Area of Telematics Engineering - University of A Coruña

`ShoppingItemList` [modo edición]

`ShoppingCart`

Desarrollo basado en componentes – Reusabilidad

- ... y para mostrar los ítems de un pedido

PA Shop

All categories ▾

Search

 (0) Orders  fernando.bellas ▾

[Back](#)

Purchase order 15

6/4/2025 - 11:48 AM

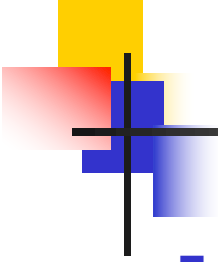
Rúe del Percebe, 13 - 12345

	Price	Quantity
2001: A Space Odyssey [Blu-ray]	€11.99	1
Blade Runner [Blu-ray]	€11.99	1
Total price: €23.98		

ShoppingItemList

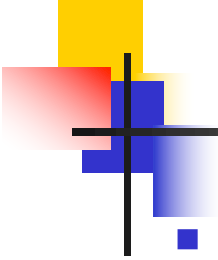
OrderDetails

PA Shop - Area of Telematics Engineering - University of A Coruña



Introducción a React

- React (<https://react.dev>) es una librería JavaScript para el desarrollo de IU mediante el paradigma del desarrollo basado en componentes
- No asume ninguna tecnología subyacente (e.g. navegador, desktop, móvil)
- Dispone de un gran ecosistema de librerías relacionadas desarrolladas por la comunidad
- Actualmente se puede usar para el desarrollo de la IU de aplicaciones web SPA (foco de la asignatura), aplicaciones web del lado servidor y aplicaciones nativas (<https://reactnative.dev>)



Introducción a React

- Un componente se puede implementar como una **función** que devuelve el markup (e.g. etiquetas HTML) del componente
- Ejemplo

```
const Welcome = () => {  
  return (  
    <div>  
      <h1>Welcome!</h1>  
    </div>  
  );  
};
```

markup

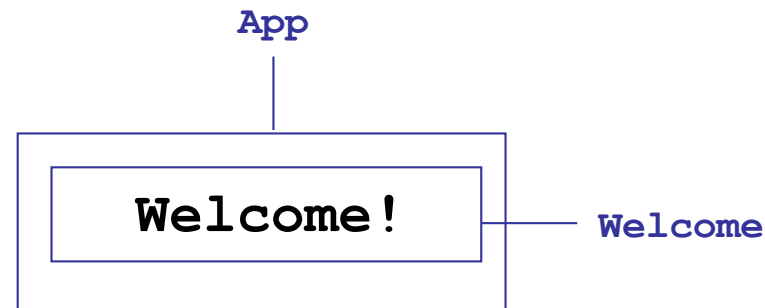
- O más sencillamente

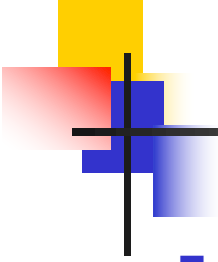
```
const Welcome = () => (  
  <div>  
    <h1>Welcome!</h1>  
  </div>  
);
```

Introducción a React

- El markup se expresa en sintaxis **JSX**
- JSX es una **extensión sintáctica a JavaScript** que permite escribir **expresiones** (en XML) para describir un conjunto de elementos (markup) devueltos, típicamente, por un componente
- Como parte del markup de un componente, se pueden incluir etiquetas de otros componentes

```
const App = () => {  
  <div>  
    <Welcome/>  
  </div>  
}
```



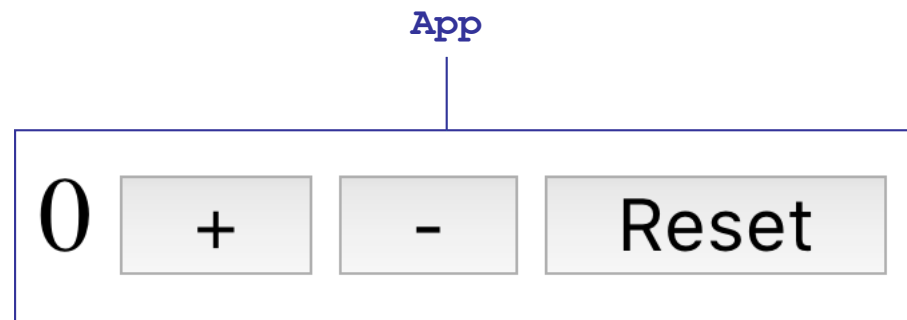


Introducción a React

- React ofrece un mecanismo para que una función que implemente un componente declare **variables de estado**
 - Las variables de estado quedan ligadas al componente función
 - El valor de una variable de estado se mantiene entre distintas “renderizaciones del componente” (ejecuciones de la función)
 - El conjunto de variables de estado de un componente define el **estado del componente**

Un primer ejemplo

- `git clone git@github.com:udc-fic-pa/pa-counter.git`
- `cd pa-counter`
- `npm install`
- `npm run dev`
- **Objetivo: crear una aplicación con un único componente (App) que permite incrementar, decrementar y resetear un contador**



App.jsx

```
import {useState} from 'react';
```

```
const App = () => {
```

Variable de estado

Función para modificarla

```
const [value, setValue] = useState(0);
```

Valor inicial

```
const handleIncrement = () => setValue(value+1);  
const handleDecrement = () => setValue(value-1);  
const handleReset = () => setValue(0);
```

Funciones anidadas

```
return (
```

Expresión JSX

```
<div>  
  {value + ' '}  
  <button onClick={handleIncrement}>+</button>  
  { ' ' }  
  <button onClick={handleDecrement}>-</button>  
  { ' ' }  
  <button onClick={handleReset}>Reset</button>  
</div>
```

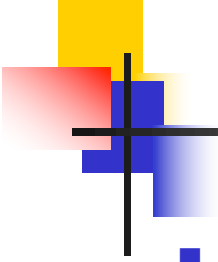
Expresión
JavaScript

Los nombres de
atributos siempre
tienen que ir en
camelCase

```
);
```

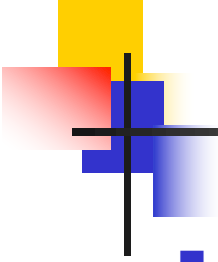
```
}
```

```
export default App;
```



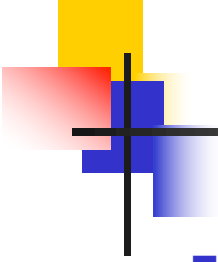
useState

- Declara una **variable de estado**
- **Recibe** el valor inicial como parámetro
- **Devuelve** un array con dos elementos
 - Primer elemento: valor actual
 - Segundo elemento: una función para actualizar el valor
 - Provoca que el componente se vuelva a renderizar (la función se vuelve a ejecutar)
 - Lo más elegante es usar **destructuring** para dar nombres naturales a la variable (**value**) que contiene el valor y a la función que permite actualizarla (**setValue**)
- En el ejemplo
 - La primera vez que se renderiza **App**, **useState** devuelve 0 en **value**
 - Cada vez que se invoca a **setValue**, el componente se vuelve a renderizar y **useState** devuelve en **value** el valor actual (el último que se estableció con **setValue**)



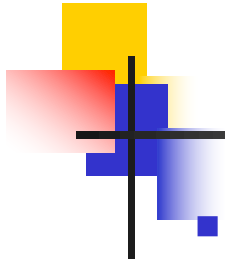
Hooks

- Las funciones que comienzan con `use` se llaman **Hooks** en la terminología React
 - Ejemplo: `useState` es un hook
- React proporciona un conjunto de Hooks predefinidos y algunas librerías también
- Los Hooks son funciones especiales, más restrictivas que otro tipo de funciones
 - **Los Hooks sólo se deben usar al comienzo de las funciones que implementan componentes (u otros Hooks), fuera de sentencias condicionales y bucles**



Más sobre JSX

- Dentro de una expresión JSX se puede incluir una expresión JavaScript mediante `{ . . . }`
 - Y dentro de una expresión JavaScript se puede volver a usar una expresión JSX y así sucesivamente
- En las etiquetas, los nombres de los atributos siempre tienen que ir en camelCase



Vite

- Para facilitar el desarrollo de frontends JavaScript se puede emplear la herramienta Vite
 - <https://vite.dev>
- Proporciona plantillas para diversos frameworks JavaScript, y en particular, para aplicaciones web SPA con React
- Creación de una aplicación
 - `npm create vite@latest my-app -- --template react`
 - Los ejemplos de este tema y el siguiente, así como el frontend de pa-shop y pa-project (plantilla del proyecto de la práctica), fueron creados de esta manera
- Extensiones de los ficheros JavaScript
 - Si un fichero usa expresiones JSX, su extensión debe ser `.jsx`
 - En otro caso, su extensión debe ser `.js`

- En la estructura de ficheros generada

- `index.html` (carpeta raíz) contiene dentro de `<body>`

```
...  
<div id="root"></div>  
...
```

- `src/main.jsx` contiene

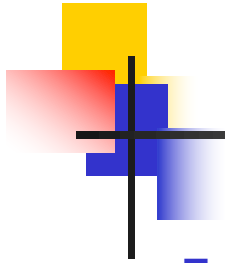
```
...  
import App from './App';  
...  
const root = ReactDOM.createRoot(document.getElementById('root'));
```

```
root.render(  
  [ - - - - - ]  
  [ <React.StrictMode> ]  
  [   <App/>   ]  
  [ </React.StrictMode> ] )
```

Muestra advertencias sobre algunos errores potenciales en el uso de React

Observación: expresión JSX

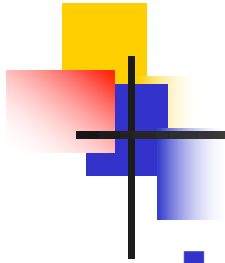
- Resultado: el componente `App` se renderiza en el `div` con id `root`



Vite

- Modo desarrollo

- `cd my-app`
- `npm run dev`
- Arranca un servidor web de desarrollo en `http://localhost:5173`
 - Se devuelve el contenido de `index.html`
 - `index.html` incluye una referencia a `/src/main.jsx`
- El resto de ficheros JavaScript se van sirviendo al navegador a medida que es necesario
- Cada vez que se modifica el código fuente del frontend, no es necesario hacer nada (el código afectado se recarga automáticamente en el navegador)
- Vite transforma los ficheros JSX a JavaScript "normal" para que se puedan ejecutar en el navegador

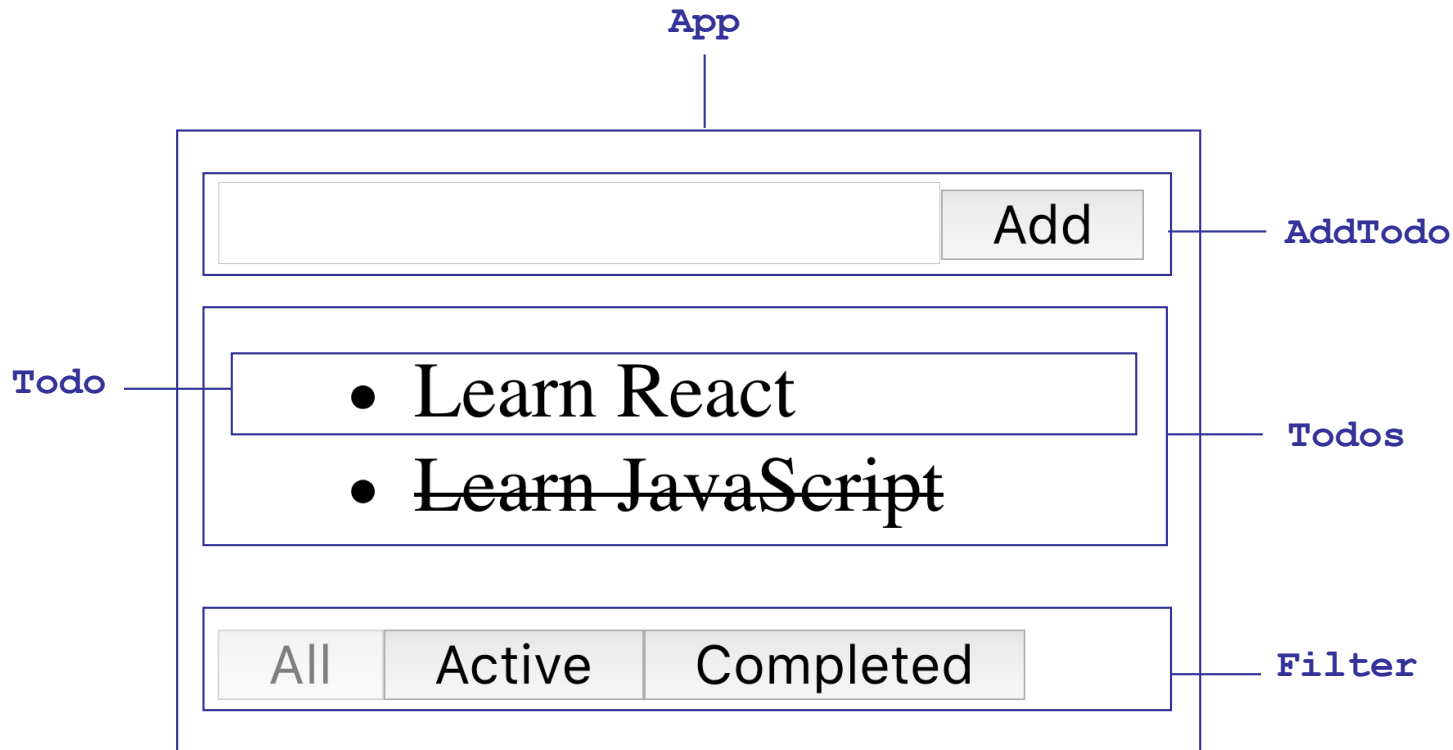


Vite

- Modo producción
 - `cd my-app`
 - `npm run build`
 - Debajo de la carpeta `dist` se crea una versión del código del frontend apta para producción
 - Los ficheros JSX se transforman a JavaScript “normal”
 - Los ficheros JavaScript también pueden necesitar transformación si se desean soportar navegadores antiguos
 - [1] Todo el código JavaScript se minimiza (e.g. se eliminan la mayor parte de los comentarios, blancos, etc.) y se incluye en un único fichero
 - [2] El código CSS también se minimiza y se incluye en un único fichero
 - `index.html` incluye una referencia a los ficheros [1] y [2]
 - El contenido de la carpeta `dist` se puede instalar en un servidor web
 - Más información
 - <https://vite.dev/guide/build.html>
 - <https://vite.dev/guide/static-deploy.html>

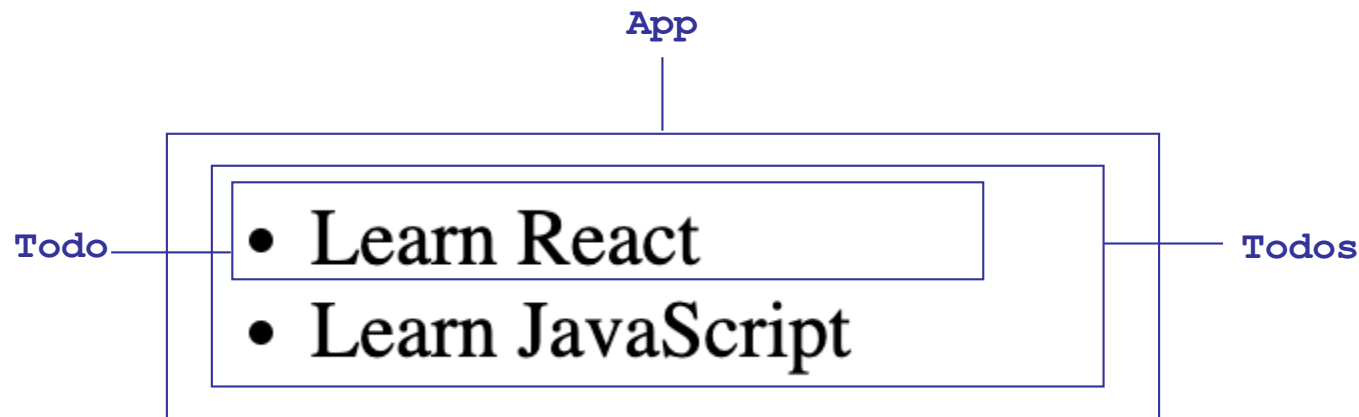
Desarrollo iterativo de un sencillo gestor de TODOs

- Desarrollaremos iterativamente un sencillo gestor de TODOs
- Por sencillez, sin backend



it-1

- `git clone git@github.com:udc-fic-pa/pa-todos.git`
- `cd pa-todos`
- `git checkout it-1`
- `npm install`
- `npm run dev`
- **Objetivo: crear una versión inicial del componente Todos**



[it-1] App.jsx

```
import Todos from './Todos';
```

```
const App = () => {
```

```
  const todos = [
```

```
    {id: 1, text: 'Learn React'},
```

```
    {id: 2, text: 'Learn JavaScript'}];
```

```
  return (
```

```
    <div>
```

```
      <Todos todos={todos}/>
```

```
    </div>
```

```
  );
```

```
}
```

```
export default App;
```

Un componente puede recibir
parámetros (propiedades)

[it-1] Todos.jsx

```
import Todo from './Todo';
```

- Los parámetros (propiedades) del componente se reciben como un objeto, con un campo por cada parámetro
- Por comodidad se puede usar **destructuring**

```
const Todos = ({ todos }) => (
```

```
  <ul>
```

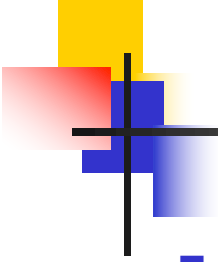
```
    { todos.map(todo => <Todo key={todo.id} todo={todo} /> ) }
```

```
  </ul>
```

```
);
```

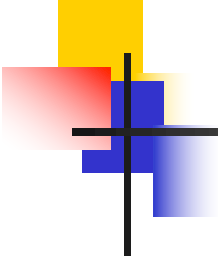
```
export default Todos;
```

Obsérvese que volvemos a insertar una expresión JSX dentro de una expresión JavaScript



[it-1] Propiedad **key**

- Cuando se renderiza una lista de un tipo de elemento (**Todo**, en el ejemplo anterior), se debe emplear la propiedad **key**
 - El valor de **key** debe ser un identificador único entre todos los hermanos (no tiene por qué ser único a nivel global)
 - En la mayor parte de los casos, el dato que renderiza cada elemento de la lista dispone de un identificador (típicamente un identificador de BD)
 - Se puede usar ese identificador como el valor de **key**

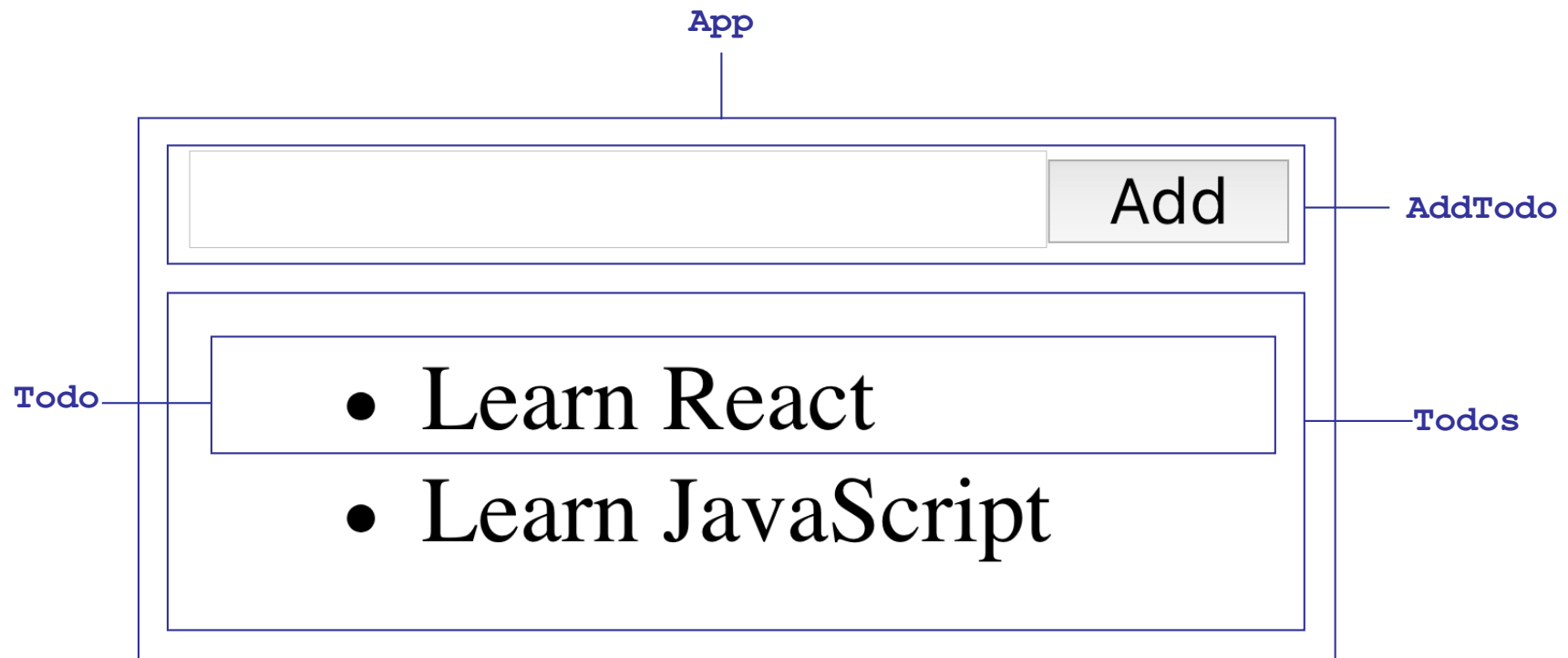


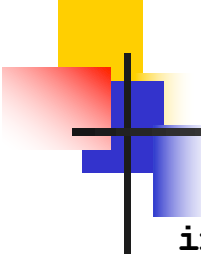
[it-1] Todo.jsx

```
const Todo = ({todo}) => (  
  <li>{todo.text}</li>  
)  
  
export default Todo;
```

it-2

- `git checkout it-2`
- **Objetivo: añadir TODOs dinámicamente**





[it-2] App.jsx

```
import {useState} from 'react';

import AddTodo from './AddTodo';
import Todos from './Todos';

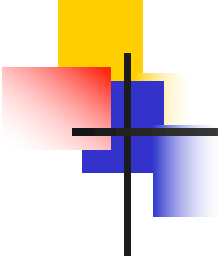
let nextTodoId = 0;

const App = () => {

  const [todos, setTodos] = useState([]);

  const todo = text => {
    return {id: nextTodoId++, text};
  }

  const handleAddTodo = text => setTodos([todo(text), ...todos]);
```



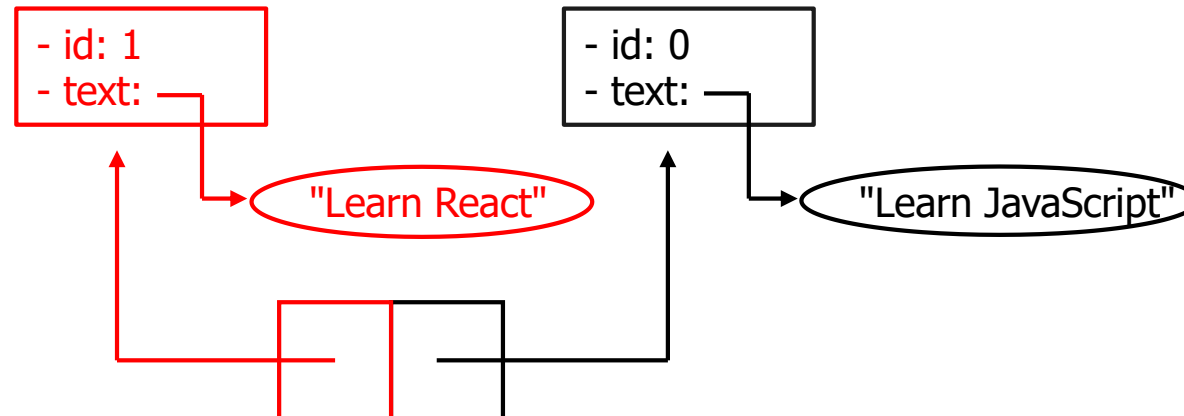
[it-2] App.jsx

```
    return (  
      <div>  
        <AddTodo onAddTodo={handleAddTodo}/>  
        <Todos todos={todos}/>  
      </div>  
    );  
  
  }  
  
  export default App;
```

[it-2] Modificación del estado

La función `handleAddTodo` se podría haber implementado modificando el array `todos`

```
const handleAddTodo = text => {  
  todos.unshift(todo(text));  
  setTodos(todos);  
}
```

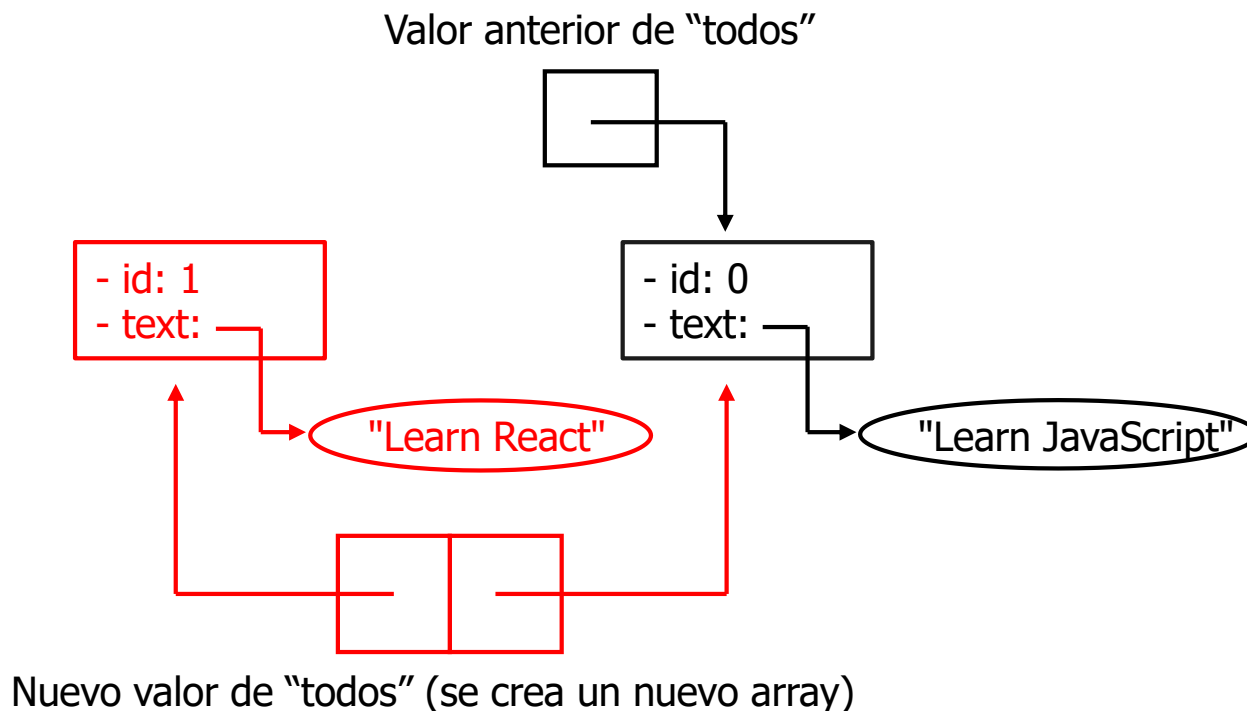


Valor de "todos" (se modifica el array, añadiendo un nuevo elemento a la cabeza)

[it-2] Modificación del estado

Sin embargo, es preferible un estilo distinto: cuando el valor de una variable sea un objeto (un array es un objeto), tratarlo como **inmutable** (no modificable)

- En el código del componente `App` se usa el **operador de propagación** ("spread operator") para crear un nuevo array



- Al final del tema, examinemos las ventajas potenciales de la inmutabilidad

[it-2] AddTodo.jsx

```
const AddTodo = ({onAddTodo}) => {
```

```
  let input;
```

```
  return (
```

```
    <form onSubmit={ (e) => {
```

```
      e.preventDefault();
```

```
      if (!input.value.trim()) {
```

```
        return;
```

```
      }
```

```
      onAddTodo(input.value.trim());
```

```
      input.value = '';
```

```
    }}>
```

```
    <input type="text" ref={node => input = node}/>
```

```
    <button type="submit">Add</button>
```

```
  </form>
```

```
);
```

```
}
```

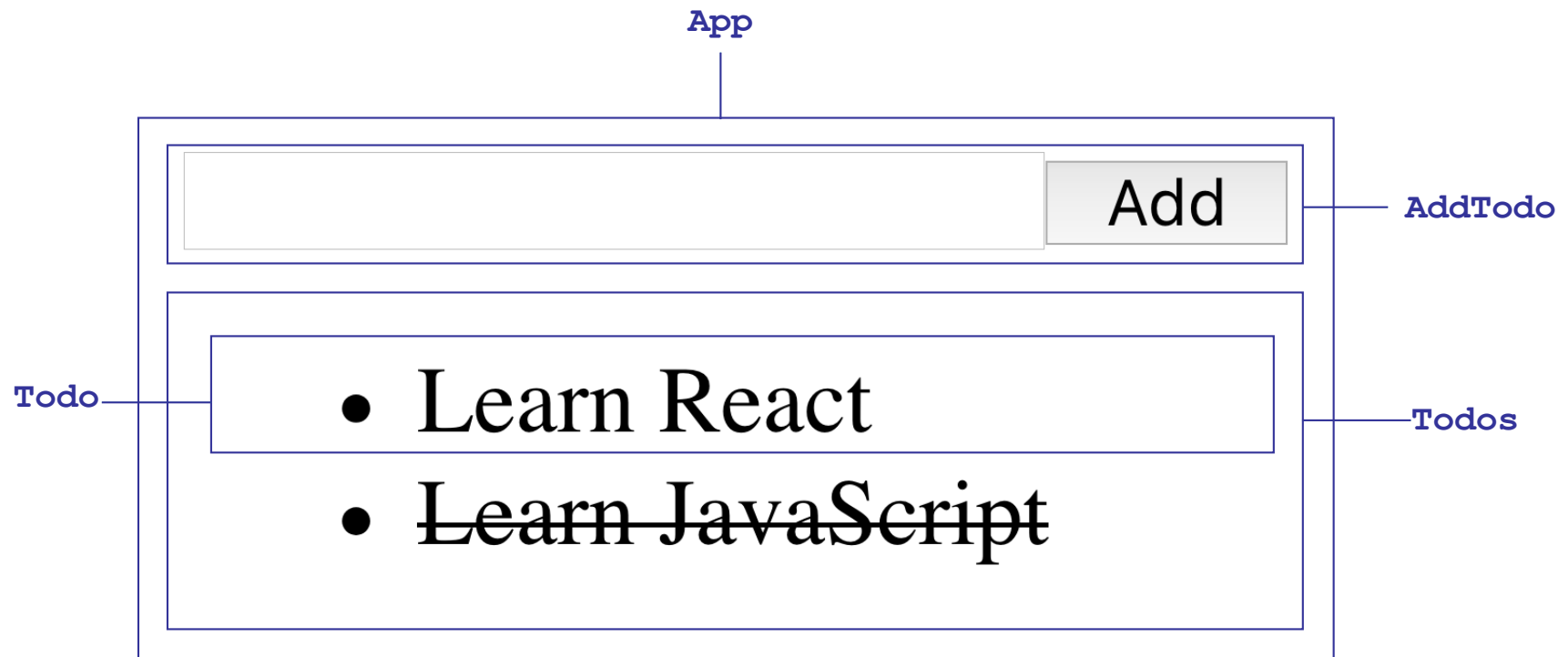
```
export default AddTodo;
```

Usamos la propiedad especial `ref` para inicializar la variable `input` con el nodo DOM correspondiente al elemento `input`



it-3

- `git checkout it-3`
- **Objetivo: permitir cambiar el estado de un TODO**



[it-3] App.jsx

```
...  
const todo = text => {  
  return {id: nextTodoId++, text, completed: false};  
}
```

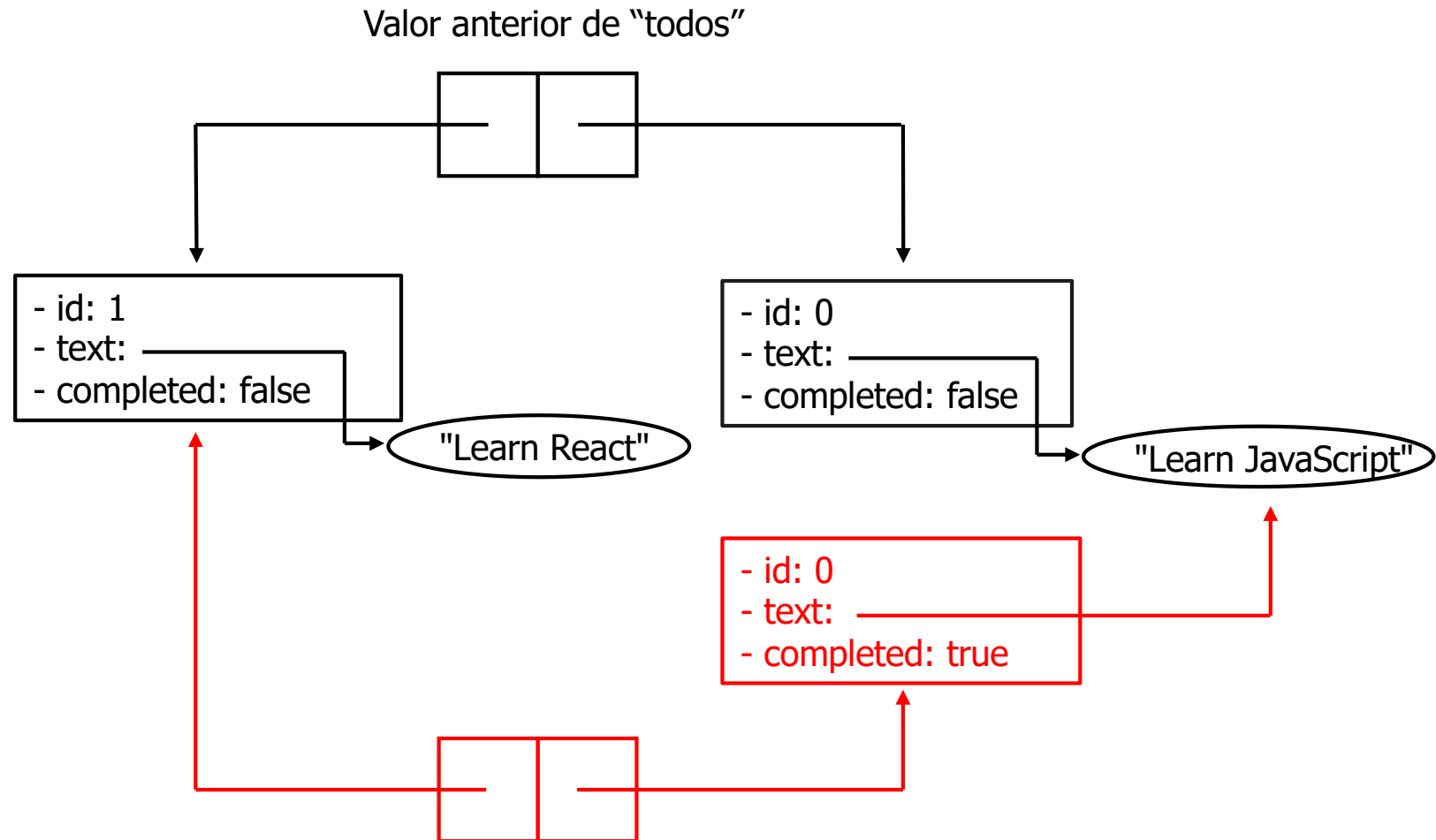
```
const handleToggleCompleted = id => {  
  const newTodos = todos.map(todo => {  
    return todo.id === id ?  
      {...todo, completed: !todo.completed} : todo;  
  });  
  setTodos(newTodos);  
}
```

Volvemos a tratar `todos` como inmutable
(siguiente diapositiva)

```
return (  
  <div>  
    <AddTodo onAddTodo={handleAddTodo}/>  
    <Todos todos={todos}  
      onToggleCompleted={handleToggleCompleted}/>  
  </div>  
);  
...
```

[it-3] App.jsx

- Obsérvese que `handleToggleCompleted` vuelve a tratar **todos** como inmutable



[it-3] Todos.jsx y Todo.jsx

■ Todos.jsx

```
...  
const Todos = ({todos, onToggleCompleted}) => (  
  <ul>  
    {todos.map(todo =>  
      <Todo key={todo.id} todo={todo}  
        onToggleCompleted={onToggleCompleted}/>  
    )}  
  </ul>  
) ;  
...
```

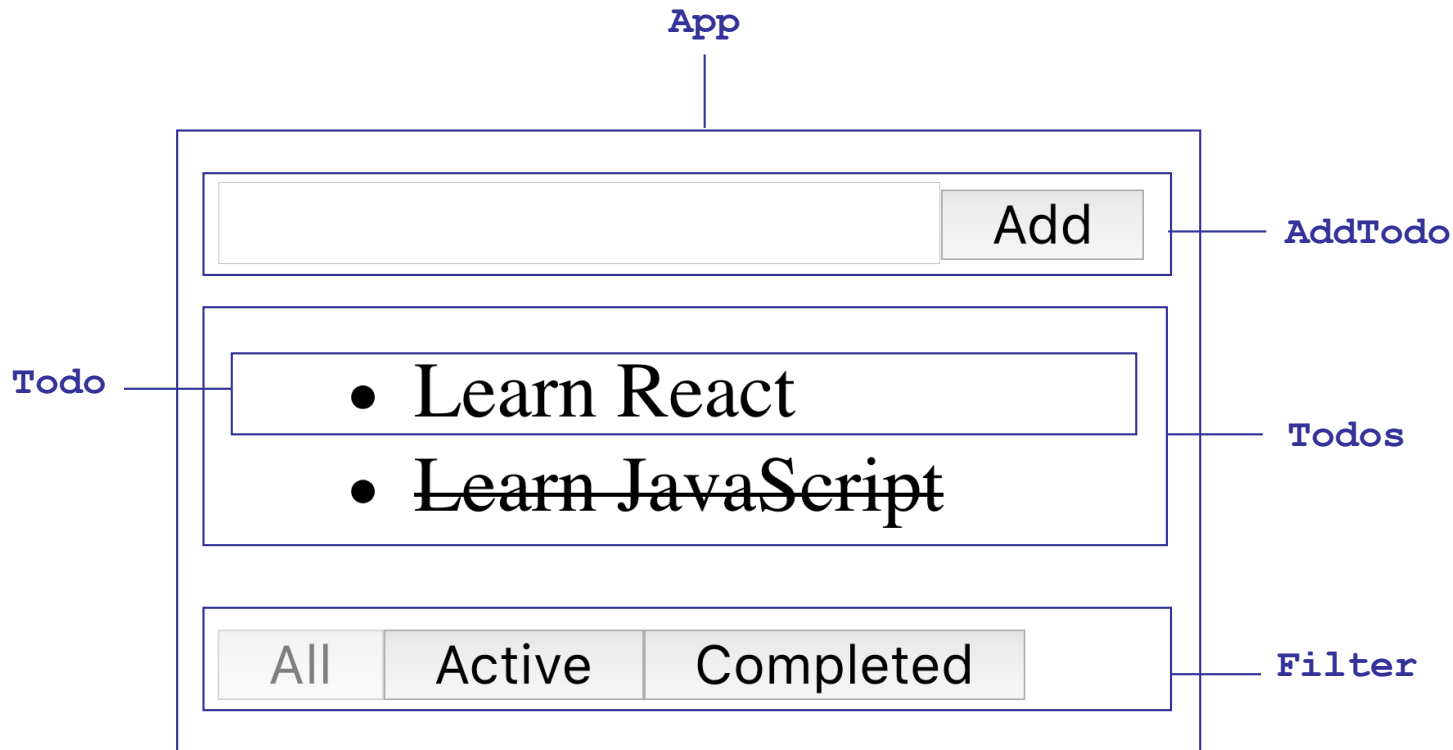
■ Todo.jsx

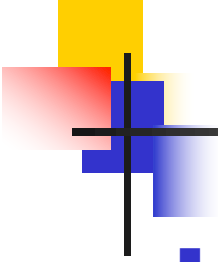
```
...  
const Todo = ({todo, onToggleCompleted}) => (  
  <li className={todo.completed ? 'completed' : ''}  
    onClick={() => onToggleCompleted(todo.id)}>{todo.text}</li>  
) ;  
...
```

- El atributo debe llamarse `className` y no `class`
- Examinamos este aspecto al final del tema

it-4

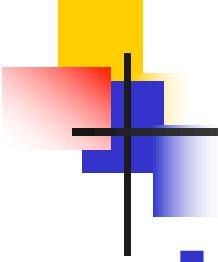
- `git checkout master`
- **Objetivo: añadir un filtro que permita mostrar todos los TODOs, sólo los activos o sólo los completados**
- Ejercicio: estudiar código fuente





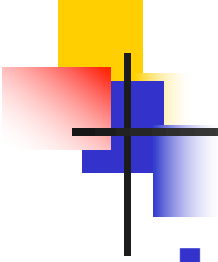
Virtual DOM

- En los ejemplos anteriores aparecen cosas que “chocan”
 - [D13] “Los nombres de atributos siempre tienen que ir en camelCase” (en referencia a los atributos de los elementos)
 - [D36] “El atributo debe llamarse `className` y no `class`”
 - Cada vez que se añade un TODO, se marca como completado/activo o se especifica un nuevo filtro, el componente **App** se vuelve a renderizar completamente
 - e.g. si se hace clic en un TODO, se vuelve a renderizar el formulario, la lista de TODOs y los filtros
- Los componentes React no renderizan directamente en el DOM del navegador, sino en una estructura de datos llamada Virtual DOM
 - Contiene el árbol de elementos renderizados por los componentes



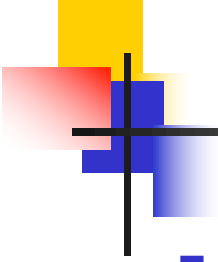
Virtual DOM

- Los elementos “HTML” (`div`, `ul`, `li`, `input`, etc.) que aparecían en los ejemplos anteriores son en realidad componentes predefinidos que corresponden a los elementos HTML del mismo nombre
 - En minúsculas
 - NOTA: los nombres de los componentes definidos por el desarrollador tienen que empezar con mayúscula
 - Atributos en camelCase
 - Con algunas diferencias en el nombrado de atributos (e.g. `className` en lugar de `class`)



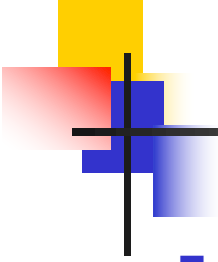
Virtual DOM

- Pero..., ¿por qué un Virtual DOM?
- Las actualizaciones al DOM del navegador son muy costosas
- React propone un **modelo de programación muy amigable** para el desarrollador
 - Cada vez que se modifica el estado de un componente, éste se vuelve a renderizar
 - Y los componentes que usa en su renderización, también
 - Resultado: la IU siempre siempre está sincronizada con los cambios de estado
- Si los componentes React renderizasen directamente en el DOM del navegador, el modelo sería muy ineficiente
 - E.g. si se hace clic en un TODO, se tendría que renderizar todo el subárbol que cuelga del `div` con `id root` (que es donde se colocaría el resultado de la renderización del componente `App`), cuando lo más óptimo sería sólo modificar el atributo `class` del `li` correspondiente a ese TODO



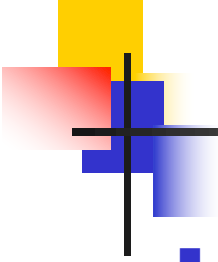
Virtual DOM

- Para poder ofrecer este modelo de programación amigable de forma eficiente
 - Cada vez que se renderiza un componente (en el Virtual DOM), React calcula la diferencia entre el resultado anterior y el nuevo
 - Aplica las diferencias al DOM del navegador
 - Ejemplo:
 - Si se hace clic en un TODO, la diferencia entre la renderización anterior de `App` y la nueva sólo es el valor del atributo `className` del elemento `li` correspondiente al nuevo TODO
 - React sólo modifica el atributo `class` del elemento HTML `li` (correspondiente al componente predefinido `li`) en el DOM del navegador



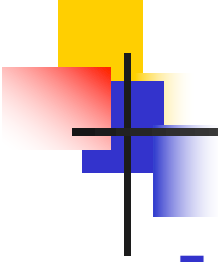
Inmutabilidad

- Siguiendo con el ejemplo anterior
 - ... Pero, el componente **App** se renderiza completamente en el Virtual DOM
- En una aplicación real, no existiría un único componente con todo el estado, sino que estaría disperso entre varios componentes (a menos que se usase una solución de gestión de estado global)
 - Esos componentes experimentarían un comportamiento similar al componente **App**
 - Se renderizan de forma eficiente en el DOM del navegador, pero provocan renderizaciones innecesarias en el Virtual DOM



Inmutabilidad

- En muchos casos, esas renderizaciones innecesarias en el Virtual DOM no causarán ningún problema de eficiencia
- Pero si lo causan, React ofrece la posibilidad de usar mecanismos de optimización
 - Estos mecanismos asumen que el estado y las propiedades del componente se tratan como inmutables (como se hace en el ejemplo anterior)
 - La idea es: se puede provocar que un componente sólo se vuelva a renderizar si el valor de las nuevas propiedades que recibe o el estado son distintos a los de la última vez
 - Como test de igualdad se utiliza la igualdad referencial: shallow comparison
 - Demo: `git checkout optimization`



Inmutabilidad

- En el próximo tema reexaminaremos la importancia de la inmutabilidad en la eficiencia
- La inmutabilidad también tiene otros beneficios potenciales (e.g. funcionalidad undo/redo)